Universe Reinsurance Platform — Database Link & Alignment Checklist

Run after every deploy or repository switch. Extracted from the *Repository Migration & Database Alignment Guide* v1.0, §7 — the guide holds the full context, seeding rules and troubleshooting.

Version 1.0 — September 2026 — Confidential, for internal IT use only. Server layout per the v2.0 specification: `/opt/universe`, user `universe`, `/opt/universe/.env`, PM2, nginx, PostgreSQL on localhost. Source: `https://github.com/iwadaya/Saudi_re`.

Checklist — is the database linked to the app, and does it align?

Work through the groups in order. "Command" runs on the application server; `psql` commands need `DATABASE_URL` in the shell — load it once with `cd /opt/universe && set -a && . ./.env && set +a` (as `universe` or root) **in a shell you will not use for `pm2` commands** (see P1). Tick every row before sign-off.

> **Deployment-kit (systemd) server?** Use these variants: C1 → `ls -lL /opt/universe/.env` = `-rw-r----- root universe` (a symlink to `/etc/universe/universe.env`); C6, R1, R2 → `sudo journalctl -u universe --no-pager -n 300 | grep …` instead of `pm2 logs`; P1 → one process, `pool.max` 50 unless `DB_POOL_MAX` is set in the env file; P2 unchanged.

7.1 Configuration — the app is told about exactly one database

#	Check	Command	Expected	Proves
C1	One env file, locked down	`ls -l /opt/universe/.env`	`-rw------- universe universe`	Secrets readable by the app user only
C2	Exactly one `DATABASE_URL`	`grep -c '^DATABASE_URL=' /opt/universe/.env`	`1`	No duplicate/overridden connection string
C3	No placeholders	`grep -c CHANGE_ME /opt/universe/.env`	`0` (grep exits 1 when the count is 0 — expected)	Real values everywhere
C4	Production mode, migrations explicit	`grep -e '^NODE_ENV=' -e '^RUN_MIGRATIONS_ON_BOOT=' /opt/universe/.env`	`NODE_ENV=production`, `RUN_MIGRATIONS_ON_BOOT=false` (or absent)	Schema changes only happen in the deploy step
C5	Seeding posture	`grep '^SEED_ON_DEPLOY=' /opt/universe/.env`	no output (absent; grep exits 1), or `SEED_ON_DEPLOY=reference` — never `1` / `reset`	No fabricated treaties can be seeded
C6	The app loaded *that* file	`sudo -u universe -H pm2 logs universe --lines 300 --nostream 2>/dev/null | grep 'startup configuration loaded' | tail -n 1`	newest line (format `0|universe | <date>: {json}`) with `"envFile":"/opt/universe/.env"`	The running process read this env file, not another

7.2 Connectivity — the database answers with the app's own credentials

#	Check	Command	Expected	Proves
D1	Connect with the app's URL	`psql "$DATABASE_URL" -c 'select 1'`	one row, `1`	Host, port, role, password and database name are right
D2	Identify what you are connected to	`psql "$DATABASE_URL" -tAc "select current_database(), current_user, inet_server_addr(), inet_server_port(), version()"`	`universe|universe|127.0.0.1|5432|PostgreSQL 16…` (your values)	You are looking at the intended database, not a stray one
D3	Postgres is up and listening locally	`sudo systemctl status postgresql --no-pager | head -3` and `ss -tlnp | grep 5432` (alternatives: `pg_lsclusters`; `psql "$DATABASE_URL" -tAc "show listen_addresses"`)	`active (running)`; only loopback sockets — `127.0.0.1:5432` and/or `[::1]:5432`, never `0.0.0.0:5432`, `*:5432` or `[::]:5432` (`pg_lsclusters` → `online`; `listen_addresses` → `localhost`)	Database service healthy and not exposed
D4	Role owns the schema	`psql "$DATABASE_URL" -tAc "select pg_get_userbyid(datdba) from pg_database where datname=current_database()"`	`universe`	Migrations can create/alter objects without superuser help

7.3 Runtime — the *running* application is connected to *that* database

#	Check	Command	Expected	Proves
R1	Boot log names the database	`sudo -u universe -H pm2 logs universe --lines 300 --nostream 2>/dev/null | grep 'DB pool connecting' | tail -n 1`	`"url":"postgresql://universe:***@localhost:5432/universe"` matching `DATABASE_URL` (password masked)	The process is using the same host/port/db you tested in D1–D2

#	Check	Command	Expected	Proves
R2	Boot sequence completed	same log, `grep -e 'database connection verified' -e 'migrations skipped' -e 'migrations complete' -e 'reference data ready' -e 'http server listening' | tail -n 12`	For the most recent start, once **per worker**: `database connection verified`, `migrations skipped`, `reference data ready` (appears twice per worker), `http server listening`. `migrations skipped` is correct — the deploy applied them; `migrations complete` appears instead only if `RUN_MIGRATIONS_ON_BOOT=true`	App verified the DB, found the schema, seeded, and is serving
R3	Deep health via the app port	`curl -sS -D - http://127.0.0.1:4000/api/health/deep`	HTTP 200, `"db":{"ok":true,"pingMs":<small>,"pool":{…,"max":<per-worker pool>,…}}`, header `X-Pool-Waiting: 0`. Under `ecosystem.config.cjs` the per-worker pool is `floor(80 / workers)` — 20 on a 4-worker box, 40 with 2 workers; 50 only when `DB_POOL_MAX` is set or the app runs outside the ecosystem file (see P1)	Live round-trip from the app to the DB, pool not saturated
R4	Deep health through nginx/TLS	`curl -sS https://<APP_DOMAIN>/api/health/deep`	same body, `"env":"production"`	Users' path reaches the same connected app
R5	DB sees the app's connections	`psql "$DATABASE_URL" -c "select usename, client_addr, state, count(*) from pg_stat_activity where datname=current_database() group by 1,2,3"`	rows with `usename = universe` (idle/active) from the app host	The connections exist in Postgres, from the expected role and host

#	Check	Command	Expected	Proves
R6	Read path: app data = DB data	`curl -sS http://127.0.0.1:4000/api/auth/users | grep -o '"username":"[^"]*"' | cut -d'"' -f4 | sort` vs `psql "$DATABASE_URL" -tAc "select username from public.uw_user where is_active order by 1"`	identical lists with the same row count. Beware the static fallback the API serves when `uw_user` is empty or missing: exactly two rows, `chief.underwriter` and `underwriter1`, with `user_id` `00000000-…-0001` / `-0002` — the same names a migrated database has, so compare the **count** with `select count(*) from public.uw_user where is_active`	The login screen is served from this database
R7	Write path: a login lands in the audit log	Log in once in the browser, then `psql "$DATABASE_URL" -c "select event_type, payload->'actor'->>'name' as username, actor as user_id, created_at from public.audit_log where event_type='LOGIN' order by created_at desc limit 3"`	top row: `username` = the account you just used (the `actor` column holds its user id), `created_at` = just now	The app writes to this database, and the clock/timezone are sane
R8	Smoke test	`BASE_URL=https://<APP_DOMAIN> /opt/universe/deploy/smoke-test.sh` (add `SMOKE_USER` / `SMOKE_PASSWORD` to test a real login)	`PASSED`	R3–R4 plus the SPA in one exit code. Its user-list step only checks that the list is non-empty, so it does **not** replace R6

7.4 Schema alignment — the code and the database agree on the migrations

#	Check	Command	Expected	Proves
S1	Nothing pending	`cd /opt/universe && sudo -u universe -H npm run migrate:status --prefix server | tail -n 3`	`Pending (0):`	Every migration file in this checkout has been applied
S2	Counts match	`psql "$DATABASE_URL" -tAc "select count(*) from public._migrations"` vs `ls /opt/universe/server/src/db/migrations/*.sql | wc -l`	equal (**149** at this commit)	No file applied twice, none missing

#	Check	Command	Expected	Proves
S3	No orphans (DB ahead of code)	`psql "$DATABASE_URL" -tAc "select filename from public._migrations" | sort > /tmp/applied.txt; ls /opt/universe/server/src/db/migrations | sort > /tmp/ondisk.txt; comm -23 /tmp/applied.txt /tmp/ondisk.txt`	**empty**	The database was never migrated by code this checkout does not contain. If a filename prints, the old repository shipped a migration the new one lacks — stop and reconcile before deploying further (`docs/migration-audit.md`)
S4	No stragglers (code ahead of DB)	`comm -13 /tmp/applied.txt /tmp/ondisk.txt`	**empty**	Same as S1, file-by-file
S5	Last migration is recent and expected	`psql "$DATABASE_URL" -c "select filename, applied_at from public._migrations order by applied_at desc limit 5"`	newest = `157_quote_offer_status_checks.sql` at this commit, `applied_at` = your deploy time	The deploy's migration step ran against this database
S6	Core tables answer	`psql "$DATABASE_URL" -c "select (select count(*) from public.contract) contracts, (select count(*) from public.quote) quotes, (select count(*) from public.claim) claims, (select count(*) from public.uw_user) users"`	your real business counts, unchanged from before the switch	The data survived; the app's tables exist

7.5 Reference-data alignment — the lookups the app expects are present

#	Check	Command	Expected	Proves
F1	Reference counts	`cd /opt/universe && sudo -u universe -H npm run seed:reference:check`	headings `Reference data (read-only check):` / `Business data (never written by this script):` ; countries ≥75, currencies ≥32, brokers ≥11, reinsurers ≥25, cedants ≥40, treaty types ≥11, classes ≥15, roles ≥9 (the check counts deactivated rows too; only a fresh database shows the exact §5.2 numbers); business rows = yours, unchanged	The reference seed reached this database

#	Check	Command	Expected	Proves
F2	Seed-only cedants and reinsurers visible	SQL in §5.2	MEDGULF, SALAMA, SAICO, Allianz Saudi Fransi and AXA Cooperative Insurance listed; reinsurer count query = 4	GCC/MENA extras present — these names come only from the reference seed
F3	No fabricated treaties	`psql "$DATABASE_URL" -tAc "select count(*) from public.contract where import_metadata->>'source'='seed:test-treaties'"`	`0`	The test portfolio was never loaded
F4	UI dropdowns	New treaty screen: Cedant, Currency, Reinsurer, Class of business, Treaty type	populated as in §5.2	The app reads the seeded lookups

7.6 Capacity and users

#	Check	Command	Expected	Proves
P1	Connection budget	`psql "$DATABASE_URL" -tAc "show max_connections"`; PM2 workers from `pm2 status`; per-worker pool from R3 `pool.max` or `sudo -u universe -H pm2 env 0 | grep DB_POOL_MAX`	workers × pool.max ≤ max_connections − 20 (e.g. 4 × 20 = 80 ≤ 80). Under PM2 the per-worker pool is what `ecosystem.config.cjs` injects: `floor(80 / workers)` (min 8), **unless `DB_POOL_MAX` was exported in the shell that ran `pm2 start` / `pm2 reload`** — then that value is used verbatim per worker (4 × 50 = 200 > 100). A `DB_POOL_MAX` line in `.env` does **not** reach the PM2 workers (they already carry the injected value); it only affects CLI scripts and the single-process systemd path. So never run `pm2 reload` from a shell that has sourced `.env`; if in doubt reload with `env -u DB_POOL_MAX pm2 reload ecosystem.config.cjs`	The cluster cannot exhaust Postgres

#	Check	Command	Expected	Proves
P2	Accounts	`cd /opt/universe/server && sudo -u universe -H node scripts/manage-user.js list`	your real users active; seeded personas (`chief.underwriter`, `retro.manager`, `underwriter1–4`) rotated or deactivated	No `demo2026` login remains
P3	Backup after the switch	`ls -lt /var/backups/universe | head -2`; weekly `scripts/verify-restore.sh` exit 0	a dump newer than the deploy. `verify-restore.sh` creates and drops a scratch database, so the role in its `DATABASE_URL` needs `CREATEDB` — grant it once with `sudo -u postgres psql -c 'ALTER ROLE universe CREATEDB'` (as `deploy/README.md` step 10 does), or run it with a superuser URL; otherwise it fails with `permission denied to create database`	Recovery point reflects the migrated schema

7.7 Sign-off summary

- ☐ §7.1 C1–C6: one env file, one `DATABASE_URL`, production mode, seeding posture correct, app loaded that file.
- ☐ §7.2 D1–D4: `psql` with the app's URL works and identifies the intended database; role owns it.
- ☐ §7.3 R1–R8: boot log, deep health (direct and via nginx), `pg_stat_activity`, user list = `uw_user`, login audited, smoke test `PASSED`.
- ☐ §7.4 S1–S6: `Pending (0)`, counts equal, no orphans, no stragglers, last migration as expected, business counts unchanged.
- ☐ §7.5 F1–F4: reference counts as in §5.2, GCC cedants visible, zero fabricated treaties, dropdowns populated.
- ☐ §7.6 P1–P3: connection budget, accounts rotated, fresh backup.

Reference — expected seed counts on a freshly migrated database

``` Reference data after seeding: countries 75 currencies 32 brokers 11 reinsurers 25 cedants 40 treaty_types 11 classes_of_business 15 roles 9 Business data (unchanged): contracts 0 quotes 0 claims 0 users 6

On an existing database expect at least these reference counts (plus the Ghana pack and user-added rows); the business rows must be your real, unchanged numbers.

### Sign-off

| Item | Value |
|---|---|
| Date / time | |
```

Item	Value
Performed by	
Deployed commit (`git log -1`)	
`migrate:status` result	Pending (0) ☐
Smoke test (R8)	PASSED ☐
Orphan / straggler check (S3, S4)	both empty ☐
Reference counts (F1)	countries …… currencies …… brokers …… reinsurers …… cedants ……
Business counts (S6)	contracts …… quotes …… claims …… users ……

Universe Reinsurance Platform — Confidential — Database Link & Alignment Checklist v1.0, September 2026.